Agenda

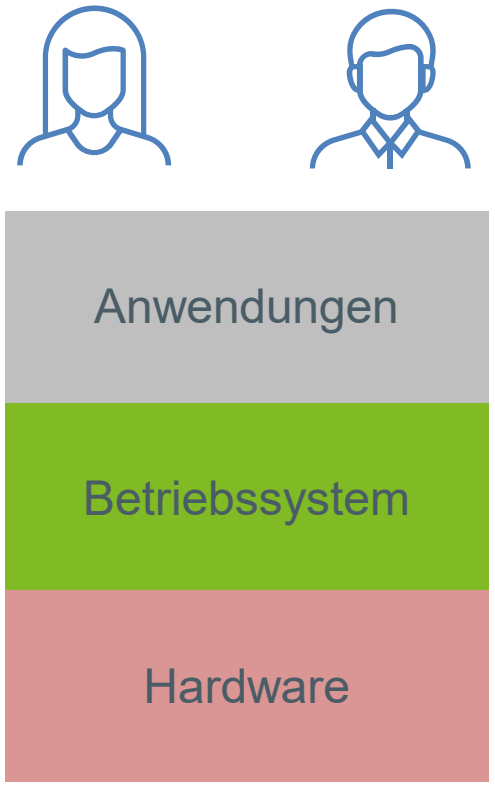
1.	Einleitung	2
2.	Versionsverwaltung	18
3.	**Betriebssysteme**	**49**
a.	Dateisystem	70
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

Betriebssysteme - Einführung

Was ist ein Betriebssystem?

Ein Betriebssystem ist ein Programm, das als Vermittler zwischen den Nutzern und ihren Anwendungen und der Hardware des Computers dient.

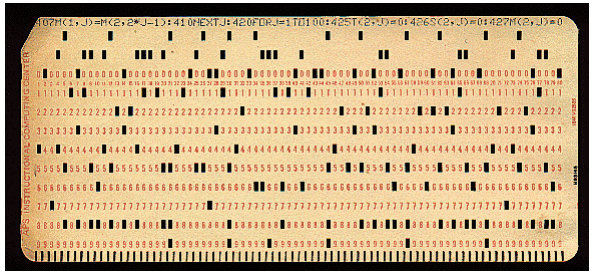
Achtung: „Nutzer“ kann auch ein anderer Computer sein.



Kurze Geschichte von Betriebssystemen - 1950

- Hardware = sehr teuer
  Menschliche Arbeitszeit = günstig
- **Ein** Benutzer
- „Schalterbrett“ als einziges Gerät; später Lochstreifen
- Software:
  Sehr maschinennah, keine „Anwendungen“ wie heute
- Ineffiziente Nutzung der CPU, da die Set-up Zeit sehr hoch war





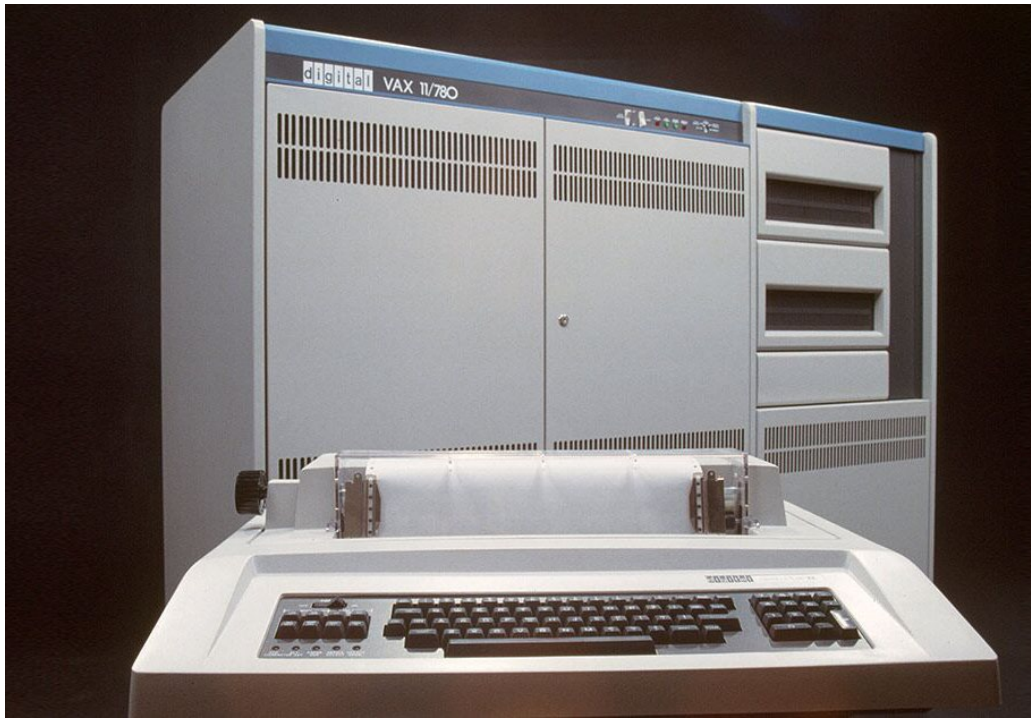
Kurze Geschichte von Betriebssystemen - 1960

- Durch die Erfindung des **Transistors** nun zuverlässigere Hardware
- **Batch** („Stapel“)-Systeme
- Reduktion der Setup-Zeit durch Zusammenfassung zu „Jobs“
- Operator bedient den Computer, ist aber nicht der Nutzer
- Ein Monitorprogramm führt Job nach Job aus
- **Programme** (ForTran oder COBOL) werden über **Lochkarten** eingegeben; nach mehreren Stunden erhält der Nutzer einen Ausdruck
- Schlechte Nutzung der CPU, da die mechanischen Geräte sehr langsam sind



Kurze Geschichte von Betriebssystemen - 1970

- Integrierte Schaltungen (**IC‘s**) erlauben kleinere und leistungsfähigere Rechner (Minicomputer wie z.B. PDP-10, VAX)
- „Time-Sharing“ : **Viele Benutzer** können „gleichzeitig“ mit dem Computer arbeiten, indem sie sich die CPU „teilen“
- Hardware wird billiger, menschliche Arbeitskraft wird teurer
- Dateisysteme, virtueller Speicher
- Software wird dediziert für einen Anwendungsfall erstellt.



Kurze Geschichte von Betriebssystemen - 1980

- Leistungsfähigere CPUs, Speicher und Geräte
- Das PC-Zeitalter beginnt
- Benutzer erhalten eigene Rechner („PC‘s“): **Ein Benutzer pro Computer**
- Hardware zu niedrigen Preisen erhältlich
- Keyboard, Maus, Bildschirm, Drucker, Disketten, Festplatten
- Kommerzielle Software für viele Anwendungszwecke erhältlich



Kurze Geschichte von Betriebssystemen - 1990

- Das **Internet-Zeitalter** beginnt
- Über das WWW erhalten Benutzer Zugriff auf Computer auf der ganzen Welt
- Anwendungen werden nicht mehr per Diskette verteilt, sondern sind auf dem Server („Web-Anwendungen“)
- Hardware ist sehr günstig
- Trennung von Nutzer-Betriebssystem und Server-Betriebssystem



Kurze Geschichte von Betriebssystemen – 2000‘er

- Mobile Systeme
- Milliarden von neuen Benutzern
- Viele Geräte pro Benutzer (PC, Tablet, Handy, Smartwatch, …)
- Leistungsfähige Hardware billiger als ein Arbeitstag eines Programmierers
- Neue Herausforderungen:
  - Sicherheit
  - Stromverbrauch
  - wenig lokaler Speicher
  - „Netz“ als einziges Gerät



Parallelrechner

- Viele CPU‘s, die eng zusammenarbeiten
- Verschiedene Programmiermodelle:
  - Verteilter vs. geteilter Speicher
  - Vektor-basiert vs. Pipelines
  - …
- Spezialisierte Anwendungen (z.B. Klimaforschung)
- Betriebssystem für zentrale Steuerung
- Betriebssystem für einzelne Rechner



El Capitan, schnellster Computer der Welt

- 43.808 HPE Cray Rechner mit insgesamt
- 11.039.616 Cores (AMD EPYC24C) und 43.808 AMD Instinct MI300A mt je 128GB
- 5,4 Petabyte Memory, Interconnect 1,2tb/s
- 1,7 ExaFlops Rechenleistung
- 700m^2 Fläche, 145 Kilometer Kabel, Stromverbrauch: 30 Megawatt

Google Pixel 10 Pro

Google Tensor G5 Octa-core
(1x3.1 GHz Cortex-X4 & 3x2.6 GHz Cortex-A720 & 4x1.92 GHz Cortex-A520)

16 GB RAM

1 TB Speicher

- 6,3 Zoll OLED 120Hz Display
- GSM / HSPA / LTE / 5G
- Wi-Fi 802.11 a/b/g/n/ac/6e/7
- Bluetooth 6.0
- NFC
- USB-C 3.2
- Satelliten SOS Service

Schnittstellen

Sensoren

- 50 Megapixel Kamera
- IMU
  - Beschleunigungssensor
  - Gyrosensor
  - Kompass
- Barometer
- GPS
- Mikrophon
- Proximity Sensor
- Licht Sensor

Aktuatoren

- Stereo Lautsprecher
- Vibrationsmotoren



Heutige Betriebssysteme



Marktanteile:

Android	Windows	iOS	OS X	Unknown	Linux
43.35%	28.24%	17.77%	5.65%	2.16%	1.5%

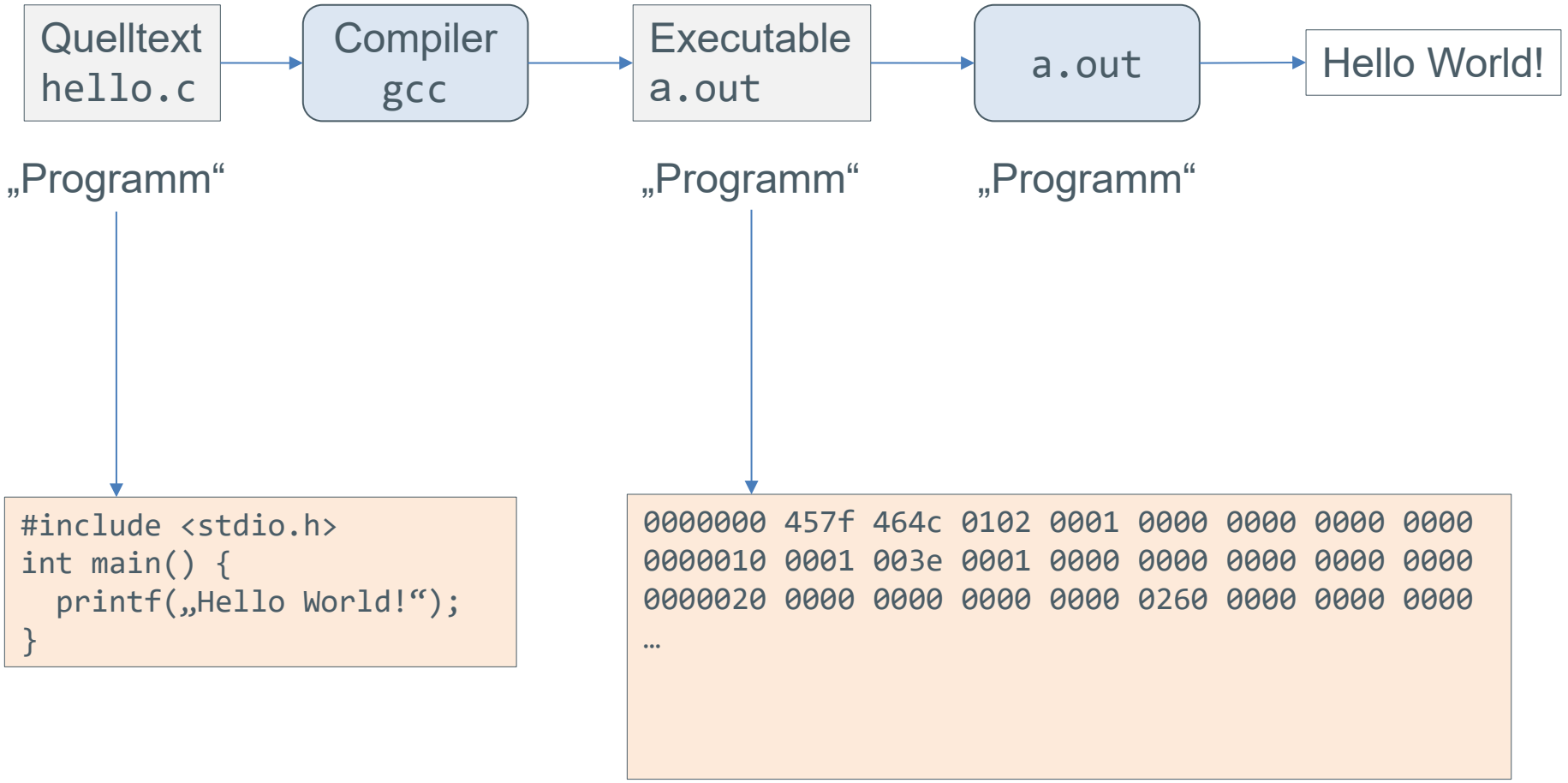


Ziele

Das Betriebssystem erfüllt folgende Ziele

- **Einfachere Ausführung der Anwendungen**
- Ermöglicht **effiziente Nutzung der Hardware**, zum Beispiel durch gemeinsame Nutzung (Sharing) von Ressourcen
- Ermöglicht es, Anwendungen auf verschiedene Hardware zu **portieren**
- **Isoliert** die Anwendungen verschiedener Nutzer voneinander und erhöht so die **Sicherheit**
- Erhöht die **Zuverlässigkeit** („Resilience“) des Systems

Erstellen und Ausführen eines Programms



```
#include <stdio.h>
int main() {
  printf(„Hello World!“);
}
```

1. Erstellen eines Quelltextes

- Benutzer startet einen Editor
- und gibt Text ein
- „Save as“ -> hello.c :
  Der Editor ruft Funktionen im Betriebssystem auf:
  - `int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);`
    Öffnet eine Datei zum Schreiben mit Permissions `0644`.
    `fd` ist der „File Descriptor“
  - `ssize_t bytes_written = write(fd, data, strlen(data));`
    Schreibt den String `data` in die Datei
  - `close(fd);`
    Schließt die Datei.

Das Betriebssystem hat also die Aufgabe mit den Geräten (zum Beispiel: Festplatte) zu kommunizieren.

```
#include <stdio.h>
int main() {
  printf(„Hello World!“);
}
```

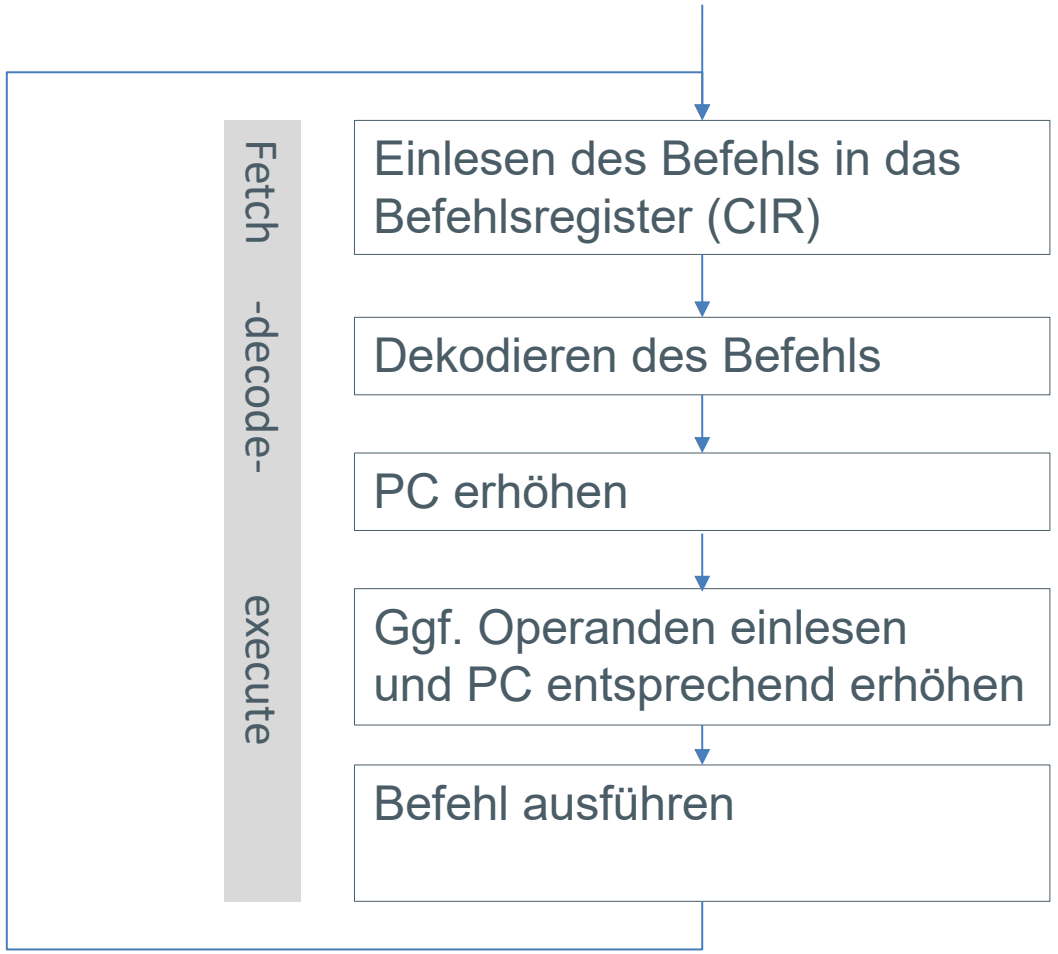
2. Kompilieren des Quelltextes

- Benutzer startet den C-Compiler: `gcc hello.c`
- Der C-Compiler öffnet die Datei hello.c zum Lesen und liest (`read`) den Inhalt ein
- Öffnet die Datei a.out zum Schreiben (`open`), schreibt (`write`) den Maschinencode als Folge von Bytes in die Datei, schließt (`close`) die Datei.

- Was ist ein Compiler?
- Ein Compiler übersetzt Programme aus einer Sprache in eine andere (in diesem Fall „C“ -> Maschinen Code).
- Schritte:
  - Einlesen der Datei und Zerlegen in Schlüsselworte, Zahlen, Bezeichner, Sonderzeichen, …
  - Prüfen der Syntax
  - Prüfen der Semantik (z.B. alle Variable, die benutzt werden, müssen deklariert sein)
  - Erstellung des Speicher-Layouts („welche Variable ist später wo im Speicher“)
  - Generierung von Assembler-Code

3. Ausführen des Programms a.out

- Das Betriebssystem
  - liest die Datei `a.out`
  - reserviert Speicher für den Code des Programms und seine Daten
- Das PC (program counter) Register wird auf die Adresse des ersten Befehls gesetzt
- Die CPU beginnt mit dem fetch-decode-execute Zyklus
- Das Betriebssystem kann die Verarbeitung des Programms unterbrechen (später dazu mehr)



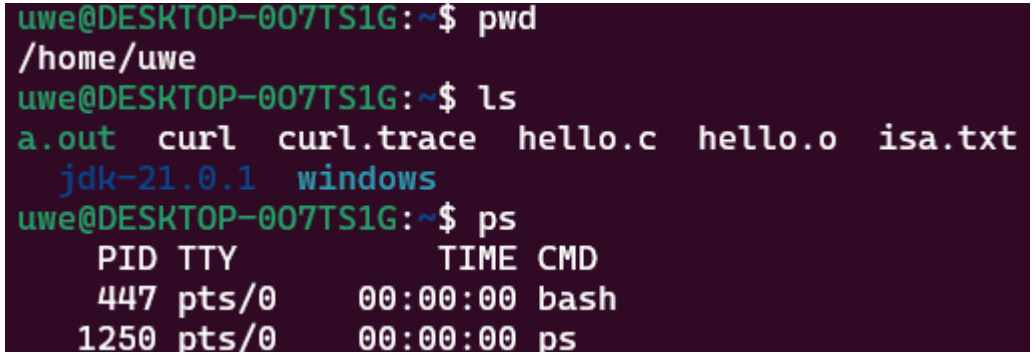
Diskussion

- Was ist der Unterschied zwischen dem Starten des Editors oder des Compilers und dem Start von `a.out`?
- Was sind die Shell bei Linux oder ‚CMD' bei Windows oder der Explorer?

Shell

- Die Shell ist eine Benutzerschnittstelle, die es erlaubt, über **Kommandos** mit dem Betriebssystem zu interagieren
- Verschiedene, gebräuchliche Shells:
  - Linux: Bourne-Shell (bs), C-Shell (csh), Korn-Shell (ksh), Bourne-Again-Shell (bash), …
  - Windows: Command Prompt (cmd), Powershell

  Die Shells besitzen üblicherweise Kommandos z.B. zum Umgang mit Dateien und Prozessen und können Programme starten.
- Shells lassen sich durch Skripte programmieren (siehe Betriebssysteme im V-Pool …)



Interaktiv



Shell-Skript

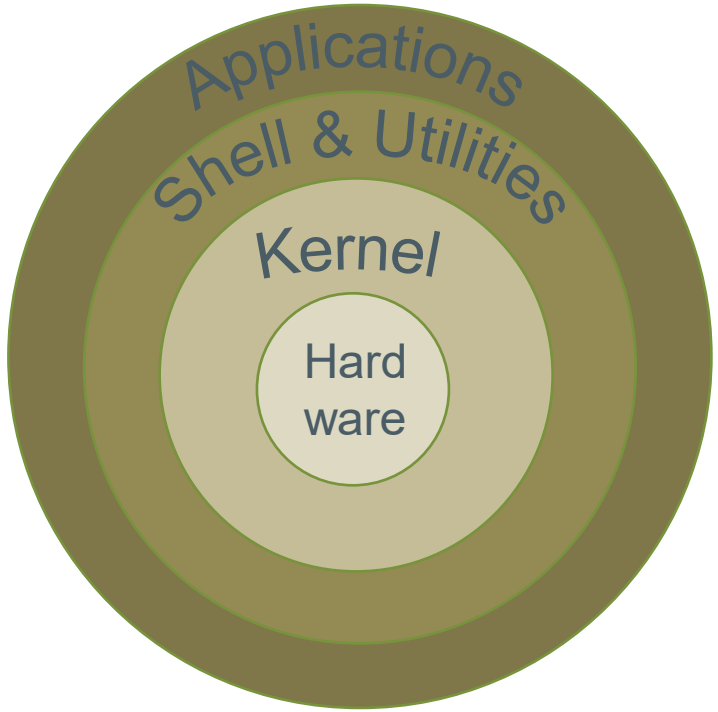
Kernel Mode – User Mode

- Der direkte Zugriff auf die Hardware durch die Anwendungen erzeugt viele Risiken
  - Sicherheit: Schadcode kann großen Schaden anrichten
  - Stabilität: Fehlnutzung durch die Anwendungen kann zu Abstürzen etc. führen
  - Ressourcen: Eine Anwendung könnte die CPU und den Speichern komplett belegen

- Ergo: Zwei Modi:
  - User Mode: Anwendungen laufen im User Mode mit eingeschränkten Rechten
  - Kernel Mode: Das Betriebssystem läuft in Kernel Mode und hat vollen Zugriff auf die Hardware

Frage: Wie kann eine Anwendung z.B. auf eine Datei zugreifen, wo sie doch im User Mode läuft?

User Mode – Kernel Mode

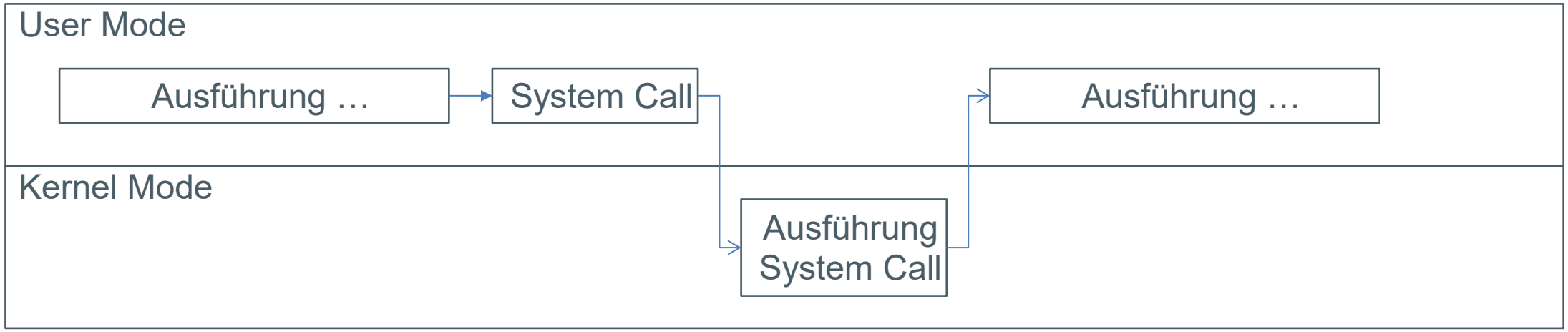


Idee: Das Betriebssystem hat einen Kern (Kernel), der von der Hardware abstrahiert.
Die Shell und andere Utilities (auch alle Shell-Kommandos) sind austauschbar.
Zu den Utilities gehören Werkzeuge, mit denen man selbst Anwendungen erstellen kann (Compiler, …).

Nur der Kernel läuft im Kernel Mode, die beiden äußeren Ringe laufen im User Mode.

Switching

- Eine Anwendung im User Mode macht einen System Call, um auf Hardware zuzugreifen (z.B. open()).
- Der System Call wird also jetzt im Betriebssystem abgearbeitet, das aber im Kernel Mode läuft und somit vollen Zugriff auf die Hardware hat.



- Ein Modus Wechsel kann mehrere hunderte CPU-Zyklen in Anspruch nehmen